



(/cl-

cookbook/)

Search...

Install an implementation

- With your package manager
- On macOS
- On Windows
- With Docker
- With the asdf-vm package manager
- With Roswell

Start a REPL

- Load and reload files
- A more ergonomic REPL: rlwrap
- A more ergonomic REPL: ICL
- The interactive debugger

Libraries

- Some terminology
- Install Quicklisp
- Install libraries

Advanced dependencies management

- Providing our own version of a library. Cloning projects.
- How to work with project-local versions of libraries

Working with projects

- Creating a new project
- How to load an existing project

More settings

See also

Credits

The Common Lisp Cookbook (/cl-cookbook/) – Getting started with Common Lisp



Learn Common Lisp efficiently in videos

(<https://www.udemy.com/course/common-lisp-programming/?couponCode=LISP-IS-NOT-DEAD>), by the Cookbook's main contributor.

Learn more (<https://github.com/vindarel/common-lisp-course-in-videos/>).



Discover a new Common Lisp and Coalton editor (/cl-cookbook/editor-support.html) for beginners: mine and a new VSCode extension (/cl-cookbook/editor-support.html#olive-new-as-of-june-2026) for Common Lisp: OLIVE .

We'll begin by presenting easy steps to install a development environment and to start a new Common Lisp project.

Install an implementation

With your package manager

If you don't know which implementation of Common Lisp to use, try SBCL (<https://www.sbcl.org/>):

```
apt-get install sbcl
```

Common Lisp is an ANSI standard but implementations can vary greatly in what they provide in addition to the standard. See Wikipedia's list of implementations (https://en.wikipedia.org/wiki/Common_Lisp#Implementations).

The following implementations are packaged for Debian and most other popular Linux distributions:

- Steel Bank Common Lisp (SBCL) (<http://www.sbcl.org/>)
- Embeddable Common Lisp (ECL) (<https://gitlab.com/embeddable-common-lisp/ecl/>), which compiles to C,
- CLISP (<https://clisp.sourceforge.io/>)

Other well-known implementations include:

- ABCL (<http://abcl.org/>), to interface with the JVM,
- ClozureCL (<https://ccl.clozure.com/>), a good implementation with very fast build times,



(/cl-

cookbook/)

Install an implementation

- With your package manager
- On macOS
- On Windows
- With Docker
- With the asdf-vm package manager
- With Roswell

Start a REPL

- Load and reload files
- A more ergonomic REPL: rlwrap
- A more ergonomic REPL: ICL
- The interactive debugger

Libraries

- Some terminology
- Install Quicklisp
- Install libraries

Advanced dependencies management

- Providing our own version of a library. Cloning projects.
- How to work with project-local versions of libraries

Working with projects

- Creating a new project
- How to load an existing project

More settings

See also

Credits

- CLASP (<https://github.com/drmeister/clasp>), that interoperates with C++ libraries using LLVM for compilation to native code,
- SICL (<https://github.com/robert-strandh/SICL>), a new and modular implementation,
- LispWorks (<http://www.lispworks.com/>) (proprietary)
- AllegroCL (<https://franz.com/products/allegrocl/>) (proprietary)

and older implementations:

- CMUCL (<https://gitlab.common-lisp.net/cmucl/cmucl>), originally developed at Carnegie Mellon University, from which SBCL is derived, and
- GNU Common Lisp (https://en.wikipedia.org/wiki/GNU_Common_Lisp)
- and there is more!

On macOS

Use homebrew (<https://brew.sh>) to install SBCL:

```
brew install sbcl
```

You can also install the Emacs editor with Homebrew, but it isn't *required* to use Common Lisp. Read below on how to use SBCL on the terminal and see the Editor Support section.

```
brew tap d12frosted/emacs-plus
brew install emacs-plus
```

On Windows

All implementations above can be installed on Windows.

SBCL is available on Chocolatey (<https://community.chocolatey.org/packages/sbcl>), albeit this is *not* an official installation method.

```
> choco install sbcl
```

You can also use plain-common-lisp (<https://github.com/pascalcombier/plain-common-lisp/>), a trivial way to get a native Common Lisp environment on Windows. It lets you install Emacs with Slime, Quicklisp and SBCL in a couple clicks: you only have to extract its archive in your workspace.

Otherwise you can install and configure Emacs yourself:

```
> choco install emacs
```

With Docker

If you already know Docker (<https://docs.docker.com>), you can get started with Common Lisp pretty quickly. The `clfoundation/cl-devel` (<https://hub.docker.com/r/clfoundation/cl-devel>) image comes with recent versions



(/cl-

cookbook/)

Install an implementation

- With your package manager
- On macOS
- On Windows
- With Docker
- With the asdf-vm package manager
- With Roswell

Start a REPL

- Load and reload files
- A more ergonomic REPL: rlwrap
- A more ergonomic REPL: ICL
- The interactive debugger

Libraries

- Some terminology
- Install Quicklisp
- Install libraries

Advanced dependencies management

- Providing our own version of a library. Cloning projects.
- How to work with project-local versions of libraries

Working with projects

- Creating a new project
- How to load an existing project

More settings

See also

Credits

of SBCL, CCL, ECL and ABCL, plus Quicklisp installed in the home (/home/cl), so than we can `ql:quickload` libraries straight away.

Docker works on GNU/Linux, Mac and Windows.

The following command will download the required image (around 1.0GB compressed), put your local sources inside the Docker image where indicated, and drop you into an SBCL REPL:

```
docker run --rm -it -v /path/to/local/code:/home/cl/common-lisp/  
source clfoundation/cl-devel:latest sbcl
```

We still want to develop using Emacs and SLIME, so we need to connect SLIME to the Lisp inside Docker. See `slime-docker` (<https://gitlab.common-lisp.net/cl-docker-images/slime-docker>), which is a library that helps on setting that up.

With the asdf-vm package manager

The asdf-vm (<http://asdf-vm.com/>) tool can be used to manage a large ecosystem of runtimes and tools.

Steel Bank Common Lisp (SBCL) (<http://www.sbcl.org/>) is available via the asdf-sbcl plugin (<https://github.com/smashedtoatoms/asdf-sbcl>).

Install it with:

```
asdf plugin-add sbcl https://github.com/smashedtoatoms/asdf-sbcl.git
```

With Roswell

Roswell (<https://github.com/roswell/roswell/wiki>) is a Common Lisp tool that is:

- an implementation manager: it makes it easy to install a Common Lisp implementation (`ros install ecl`), an exact version of an implementation (`ros install sbcl/1.2.0`), to change the default one being used (`ros use ecl`),
- a scripting environment (helps to run Lisp from the shell, to get the command line arguments,...),
- a programs installer,
- a testing environment (to run tests, including on popular Continuous Integration platforms),
- a building utility (to build images and executables in a portable way).

You'll find several ways of installation on its wiki (Debian package, Windows installer, Brew/Linux Brew,...).

Start a REPL

Just launch the implementation executable on the command line to enter the REPL (Read Eval Print Loop), i.e. the interactive interpreter.



(/cl-

cookbook/)

Install an implementation

- With your package manager
- On macOS
- On Windows
- With Docker
- With the asdf-vm package manager
- With Roswell

Start a REPL

- Load and reload files
- A more ergonomic REPL: rlwrap
- A more ergonomic REPL: ICL
- The interactive debugger

Libraries

- Some terminology
- Install Quicklisp
- Install libraries

Advanced dependencies management

- Providing our own version of a library. Cloning projects.
- How to work with project-local versions of libraries

Working with projects

- Creating a new project
- How to load an existing project

More settings

See also

Credits

Quit with (quit) or `ctr-d` (on some implementations).

Here is a sample session: we start the `sbcl` binary, we see a startup message, we are landed in a Lisp prompt (the `*`), we enter a lisp form, then we quit.

```
user@debian:~$ sbcl
This is SBCL 2.1.11.debian, an implementation of ANSI Common Lis
p.
More information about SBCL is available at <http://www.sbcl.or
g/>.

SBCL is free software, provided as is, with absolutely no warran
ty.
It is mostly in the public domain; some portions are provided un
der
BSD-style licenses. See the CREDITS and COPYING files in the
distribution for more information.
* (+ 1 2)

3
* (quit)
user@debian:~$
```

Load and reload files

You just evaluated Lisp code in a REPL, congratulations. Of course, you'll want to write code to a `.lisp` file.

Use `sbcl --load myfile.lisp` to load, compile and run this file. After the top-level commands were run, the Lisp process doesn't exit and it gives you a REPL, so you can keep working.

Once you edit your `.lisp` file, you don't need to quit and call the SBCL command to `--load` your file again. You can simply `load` it from within the REPL:

```
$ sbcl --load myfile.lisp

a... bunch... of... awesome... stuff

* (load "myfile.lisp")
```

Did you see this `*` bit? It's the default Lisp prompt in a terminal. In our editors, we usually see `CL-USER>`, denoting the current package.

Speaking of editors: of course, we can have a much more interactive workflow with a good editor setup. But you can already work like this.

A more ergonomic REPL: rlwrap

The REPL is not very ergonomic out of the box, at least with SBCL: the arrow keys do not work in order to recall history (the previous commands entered), you don't have completion of built-in Lisp functions, etc.



(/cl-

cookbook/)

Install an implementation

- With your package manager
- On macOS
- On Windows
- With Docker
- With the asdf-vm package manager
- With Roswell

Start a REPL

- Load and reload files
- A more ergonomic REPL: rlwrap
- A more ergonomic REPL: ICL
- The interactive debugger

Libraries

- Some terminology
- Install Quicklisp
- Install libraries

Advanced dependencies management

- Providing our own version of a library. Cloning projects.
- How to work with project-local versions of libraries

Working with projects

- Creating a new project
- How to load an existing project

More settings

See also

Credits

You can slightly enhance its functionality by installing and using `rlwrap`.

Operating system command

Linux (Debian) `apt-get install rlwrap`
 macOS `brew install rlwrap`
 Windows `choco install rlwrap`

Then invoke it like so:

```
rlwrap sbcl
```

This brings *some* quality of life to a bare-bones terminal REPL, but we'll setup our editor to offer a much better editing experience instead of working in such a terminal REPL. See editor-support (editor-support.html).

TIP: The CLISP implementation has a better default REPL for the terminal (readline capabilities, completion of symbols). You can even use `clisp -on-error abort` to have error messages without the debugger. It's handy to try things out, but we recommend to set-up your editor and to use SBCL or CCL.

TIP: By adding the `-c` switch to `rlwrap`, you can autocomplete file names.

A more ergonomic REPL: ICL

ICL (<https://github.com/atgreen/icl>), Interactive Common Lisp, is an enhanced REPL for the terminal. It works with any implementation you have on your system. It brings a *lot* of nice features. While it doesn't replace a good editor, it can actually complement it.

ICL provides:

- a **modern interactive experience** with readline-style editing: persistent history, tab completion, and an extensible command system.
 - it is better than `rlwrap` because it knows about Lisp code. It will offer TAB-completion for Lisp built-ins and for your code.
- it works in the terminal,
- it also works in a web browser, where
 - we can browse packages and systems,
 - we can click and browse documentation
 - we can use its interactive data inspector
 - we can display certain variables' content visually (JSON string, images in byte arrays...)
 - we can define our own visualization functions
 - and we can watch the representation update as the variables change
 - we can launch an interactive Speedscope flame graph for performance profiling,
 - we can run a code-coverage report (with `sb-cover`, SBCL only)
 - and surely more.



(/cl-

cookbook/)

Install an implementation

- With your package manager
- On macOS
- On Windows
- With Docker
- With the asdf-vm package manager
- With Roswell

Start a REPL

- Load and reload files
- A more ergonomic REPL: rlwrap
- A more ergonomic REPL: ICL
- The interactive debugger

Libraries

- Some terminology
- Install Quicklisp
- Install libraries

Advanced dependencies management

- Providing our own version of a library. Cloning projects.
- How to work with project-local versions of libraries

Working with projects

- Creating a new project
- How to load an existing project

More settings

See also

Credits

- it can also be started from your editor (Emacs) and be connected with all the nice features of the browser window.

Also note that ICL is beginner friendly. It doesn't have an interactive debugger. Instead, it handles any error, prints the error message, and allows the programmer to see the backtrace with the `,bt` command.

You can install it with GNU/Linux pre-built binaries and packages (.deb, .rpm), on Windows with a zip, an exe or a MSI installer. See the GitHub releases (<https://github.com/atgreen/icl/releases>). It is also easy to build.

Once you have `icl` in your path, start it:

```
$ icl
COMMON-LISP-USER>
```

you can also `load` a file before starting the REPL with the `--load / -l` option:

```
$ icl -l mycode.lisp
your... output...
COMMON-LISP-USER> (your-function-is-available)
```

Have fun!

A similar project is `cl-repl`, see the editors section.

ICL is a project coded with the help of LLMs.

The interactive debugger

Lisp is interactive by nature, so in case of an error we enter the debugger. This can be annoying in certain cases, so you might want to use SBCL's `--disable-debugger` option.

Also, note that `icl` (see above) doesn't have an interactive debugger, it prints the error message and allows you to print the backtrace.

Libraries

Common Lisp has thousands of libraries available under a free software license. See:

- the awesome-cl (<https://github.com/CodyReichert/awesome-cl>) list, a curated list of libraries.
- Quickdocs (<http://quickdocs.org/>) - the library documentation hosting for CL.
- Cliki (<http://www.cliki.net/>), the Common Lisp wiki.

Some terminology

- In the Common Lisp world, a **package** is a way of grouping symbols together and of providing encapsulation. It is similar to a C++ namespace, a Python module or a Java package.



(/cl-

cookbook/)

Install an implementation

- With your package manager
- On macOS
- On Windows
- With Docker
- With the asdf-vm package manager
- With Roswell

Start a REPL

- Load and reload files
- A more ergonomic REPL: rlwrap
- A more ergonomic REPL: ICL
- The interactive debugger

Libraries

- Some terminology
- Install Quicklisp
- Install libraries

Advanced dependencies management

- Providing our own version of a library. Cloning projects.
- How to work with project-local versions of libraries

Working with projects

- Creating a new project
- How to load an existing project

More settings

See also

Credits

- A **system** is a collection of CL source files bundled with an .asd file which tells how to compile and load them. There is often a one-to-one relationship between systems and packages, but this is in no way mandatory. A system may declare a dependency on other systems. Systems are managed by ASDF (<https://common-lisp.net/project/asdf/asdf.html>) (Another System Definition Facility), which offers functionalities similar to those of `make` and `ld.so`, and has become a de facto standard.
- A Common Lisp library or project typically consists of one or several ASDF systems (and is distributed as one Quicklisp project).

Install Quicklisp

Quicklisp (<https://www.quicklisp.org/beta/>) is more than a package manager, it is also a central repository (a *dist*) that ensures that all libraries build together.

SEE ALSO: the new OCICL (<https://github.com/ocicl/ocicl/>) package manager. It provides a command-line tool, it works with project-local dependencies (read more below), it uses HTTPS by default and it is a repository independent from Quicklisp.

It provides its own *dist* but it is also possible to build our own.

To install it, we can either:

1- run this command, anywhere:

```
curl -O https://beta.quicklisp.org/quicklisp.lisp
```

and enter a Lisp REPL and load this file:

```
sbcl --load quicklisp.lisp
```

or

2- install the Debian package:

```
apt-get install cl-quicklisp
```

and load it, from a REPL:

```
(load "/usr/share/common-lisp/source/quicklisp/quicklisp.lisp")
```

Then, in both cases, still from the REPL:

```
(quicklisp-quickstart:install)
```

This will create the `~/quicklisp/` directory, where Quicklisp will maintain its state and downloaded projects.



(/cl-

cookbook/)

Install an implementation

- With your package manager
- On macOS
- On Windows
- With Docker
- With the asdf-vm package manager
- With Roswell

Start a REPL

- Load and reload files
- A more ergonomic REPL: rlwrap
- A more ergonomic REPL: ICL
- The interactive debugger

Libraries

- Some terminology
- Install Quicklisp
- Install libraries

Advanced dependencies management

- Providing our own version of a library. Cloning projects.
- How to work with project-local versions of libraries

Working with projects

- Creating a new project
- How to load an existing project

More settings

See also

Credits

If you wish, you can install Quicklisp to a different location. For instance, to install it to a hidden folder on Unix systems:

```
;; optional
(quicklisp-quickstart:install :path "~/.quicklisp")
```

Finally, in order to always load Quicklisp when you start a new Lisp session, run:

```
(ql:add-to-init-file)
```

this adds the right stuff to the init file of your CL implementation. Otherwise, you have to run `(load "~/.quicklisp/setup.lisp")` in every session if you want to use Quicklisp or any of the libraries installed through it.

It adds the following in your (for example) `~/.sbclrc`:

```
#-quicklisp
(let ((quicklisp-init (merge-pathnames
                        "quicklisp/setup.lisp"
                        (user-homedir-pathname))))
  (when (probe-file quicklisp-init)
    (load quicklisp-init)))
```

Install libraries

In the REPL:

```
(ql:quickload "system-name")
```

For example, this installs the “str (<https://github.com/vindarel/cl-str/>)” string manipulation library:

```
(ql:quickload "str")
```

and voilà. You can use it right away:

```
(str:title-case "HELLO LISP!")
```

SEE MORE: To understand the `package:a-symbol` notation, read the packages page ([packages.html](https://edocl.github.io/cl-ppcre/)), section “Accessing symbols from a package”.

We can install more than one library at once. Here we install `cl-ppcre` (<https://edocl.github.io/cl-ppcre/>) for regular-expressions, and `Alexandria` (<https://alexandria.common-lisp.dev/draft/alexandria.html>), a utility library:

```
(ql:quickload '("str" "cl-ppcre" "alexandria"))
```




(/cl-

cookbook/)

Install an implementation

- With your package manager
- On macOS
- On Windows
- With Docker
- With the asdf-vm package manager
- With Roswell

Start a REPL

- Load and reload files
- A more ergonomic REPL: rlwrap
- A more ergonomic REPL: ICL
- The interactive debugger

Libraries

- Some terminology
- Install Quicklisp
- Install libraries

Advanced dependencies management

- Providing our own version of a library. Cloning projects.
- How to work with project-local versions of libraries

Working with projects

- Creating a new project
- How to load an existing project

More settings

See also

Credits

Anytime you want to use a third-party library in your Lisp REPL, you can run this `ql:quickload` command. It will not hit the network a second time if it finds that the library is already installed on your file system. Libraries are by default installed in `~/quicklisp/dists/quicklisp/`.

Note also that dozens of Common Lisp libraries are packaged in Debian. The package names usually begin with the `cl-` prefix (use `apt-cache search --names-only "^cl-.*"` to list them all).

For example, in order to use the `cl-ppcre` library, one should first install the `cl-ppcre` package.

Then, in SBCL, it can be used like this:

```
(require "asdf")
(require "cl-ppcre")
(cl-ppcre:regex-replace "fo+" "foo bar" "frob")
```

Here we pretend we don't have Quicklisp installed and we use `require` to load a module that is available on the file system. In doubt, you can use `ql:quickload`.

See Quicklisp's documentation for more commands. For instance, see how to upgrade or rollback your Quicklisp's distribution.

Advanced dependencies management

You can drop Common Lisp projects into any of these folders:

- `~/quicklisp/local-projects`
- `~/common-lisp`,
- `~/.local/share/common-lisp/source`,

A library installed here is automatically available for every project.

For a complete list, see the values of:

```
(asdf/source-registry:default-user-source-registry)
```

and

```
asdf:*central-registry*
```

`*central-registry*` is a top-level variable inside the `asdf` package written with so-called “`*earmuffs*`”. They are a useful convention, see the Variables (variables.html) chapter.



(/cl-

cookbook/)

Install an implementation

- With your package manager
- On macOS
- On Windows
- With Docker
- With the asdf-vm package manager
- With Roswell

Start a REPL

- Load and reload files
- A more ergonomic REPL: rlwrap
- A more ergonomic REPL: ICL
- The interactive debugger

Libraries

- Some terminology
- Install Quicklisp
- Install libraries

Advanced dependencies management

- Providing our own version of a library. Cloning projects.
- How to work with project-local versions of libraries

Working with projects

- Creating a new project
- How to load an existing project

More settings

See also

Credits

Providing our own version of a library. Cloning projects.

Given the property above, we can clone any library into the `~/quicklisp/local-projects/` directory and it will be found by ASDF (and Quicklisp) and available right-away:

```
(asdf:load-system "system")
```

or

```
(ql:quickload "system")
```

The practical difference between the two is that `ql:quickload` first tries to fetch the system from the Internet if it is not already installed.

Note that symlinks in local-projects to another location of your liking work too.

How to work with project-local versions of libraries

If we need libraries to be installed locally, for only one project, or in order to easily ship a list of dependencies with an application, we can use Qlot (<https://github.com/fukamachi/qlot>) or ocicl (<https://github.com/ocicl/ocicl/>).

Both projects provide a command-line tool, allow to pin dependencies in a lock file (`qlfile.lock` and `ocicl.csv`) and commands to install and manage dependencies locally, without touching your global Quicklisp installation (`qlot install`, `ocicl install`).

Of course you are recognizing the dependency management style that is predominant out there in other ecosystems: `npm`, `pip` etc. However we are only presenting these tools now because they are far less needed in Common Lisp.

NOTE: newcomers don't need to worry about project-local dependencies yet. We can go a long way with global Quicklisp dependencies and `quicklisp/local-projects/`. Choose the simplest tool for you (and it's surely Quicklisp).

The reasons are at least two fold: the language (and its implementations) is very stable, for example implementations *never* introduce breaking changes of *lisp syntax*. They introduce minor breaking changes from time to time that only hardcore users will notice. Second, the ecosystem is very stable too and usually *library authors play the game of stability*. We saw deprecation warnings staying for 12 (twelve) years. All in all, a newcomer in Common Lisp land doesn't need to worry too much if at all about project-local dependencies.

Let's mention another couple tools for the task.



(/cl-

cookbook/)

Install an implementation

- With your package manager
- On macOS
- On Windows
- With Docker
- With the asdf-vm package manager
- With Roswell

Start a REPL

- Load and reload files
- A more ergonomic REPL: rlwrap
- A more ergonomic REPL: ICL
- The interactive debugger

Libraries

- Some terminology
- Install Quicklisp
- Install libraries

Advanced dependencies management

- Providing our own version of a library. Cloning projects.
- How to work with project-local versions of libraries

Working with projects

- Creating a new project
- How to load an existing project

More settings

See also

Credits

Quicklisp also provides Quicklisp bundles

(<https://www.quicklisp.org/beta/bundles.html>). They are self-contained sets of systems that are exported from Quicklisp and loadable without involving Quicklisp.

At last, there's Quicklisp controller (<https://github.com/quicklisp/quicklisp-controller>) to help us build *dists*.

Working with projects

Now that we have Quicklisp and our editor ready, we can start writing Lisp code in a file and interacting with the REPL.

But if we want to work with an existing project or create a new one, how do we proceed? What's the right sequence of `defpackage` ? What should we put in the `.asd` file? How do we load the project into the REPL ?

Creating a new project

Some project builders help to scaffold the project structure. We like `cl-project` (<https://github.com/fukamachi/cl-project>), which also sets up a test skeleton.

In short:

```
(ql:quickload "cl-project")
(cl-project:make-project #P"./path-to-project/root/")
```

will create a directory structure like this:

```
|-- my-project.asd
|-- my-project-test.asd
|-- README.markdown
|-- README.org
|-- src
|   |-- my-project.lisp
|-- tests
|   |-- my-project.lisp
```

where `my-project.asd` resembles this:



(/cl-

cookbook/)

Install an implementation

- With your package manager
- On macOS
- On Windows
- With Docker
- With the asdf-vm package manager
- With Roswell

Start a REPL

- Load and reload files
- A more ergonomic REPL: rlwrap
- A more ergonomic REPL: ICL
- The interactive debugger

Libraries

- Some terminology
- Install Quicklisp
- Install libraries

Advanced dependencies management

- Providing our own version of a library. Cloning projects.
- How to work with project-local versions of libraries

Working with projects

- Creating a new project
- How to load an existing project

More settings

See also

Credits

```
(asdf:defsystem "my-project"
  :version "0.1.0"
  :author ""
  :license ""
  :depends-on () ;; <== list of Quicklisp dependencies
  :components ((:module "src"
                     :components
                     ((:file "my-project"))))
  :description ""
  :long-description
  #.(read-file-string
      (subpathname *load-pathname* "README.markdown"))
  :in-order-to ((test-op (test-op "my-project-test"))))
```

and src/my-project.lisp this:

```
(defpackage footest
  (:use :cl))
(in-package :footest)
```

- ASDF documentation: defining a system with defsystem (<https://common-lisp.net/project/asdf/asdf.html#Defining-systems-with-defsystem>)

How to load an existing project

You have created a new project, or you have an existing one, and you want to work with it in the REPL, but Quicklisp doesn't know about it. What do you do?

Well first, if you create it or clone it into one of `~/common-lisp`, `~/local/share/common-lisp/source/` or `~/quicklisp/local-projects`, you'll be able to `(ql:quickload ...)` it with no further ado.

Otherwise you'll need to compile and load its system definition (`.asd`) first. In SLIME with the `slime-asdf` contrib loaded, type `C-c C-k` (*slime-compile-and-load-file*) in the `.asd`, then you can `(ql:quickload ...)` it.

You can use `(asdf:load-asd "my-project.asd")` programmatically instead of `C-c C-k`.

Usually you want to "enter" the system in the REPL at this stage:

```
(in-package :my-project)
```

Lastly, you can compile or eval the sources (`my-project.lisp`) with `C-c C-k` or `C-c C-c` (*slime-compile-defun*) in a form, and see its result in the REPL.

Another solution is to use ASDF's list of known projects:

```
;; startup file like ~/.sbclrc
(pushnew "~/to/project/" asdf:*central-registry* :test #'equal)
```



(/cl-

cookbook/)

Install an implementation

- With your package manager
- On macOS
- On Windows
- With Docker
- With the asdf-vm package manager
- With Roswell

Start a REPL

- Load and reload files
- A more ergonomic REPL: rlwrap
- A more ergonomic REPL: ICL
- The interactive debugger

Libraries

- Some terminology
- Install Quicklisp
- Install libraries

Advanced dependencies management

- Providing our own version of a library. Cloning projects.
- How to work with project-local versions of libraries

Working with projects

- Creating a new project
- How to load an existing project

More settings

See also

Credits

and since ASDF is integrated into Quicklisp, we can `quickload` our project right away.

Happy hacking !

More settings

You might want to set SBCL's default encoding format to utf-8:

```
(setf sb-impl::*default-external-format* :utf-8)
```

You can add this to your `~/.sbclrc`.

If you dislike the REPL printing all symbols uppercase, add this:

```
(setf *print-case* :downcase)
```

Warning: This might break the behaviour of some packages like happened with Mito (<https://github.com/fukamachi/mito/issues/45>). Avoid doing this in production.

See also

- OCICL (<https://github.com/ocicl/ocicl/>) - a modern alternative to quicklisp for ASDF system distribution and management, code linting, project scaffolding.
- cl-cookieproject (<https://github.com/vindarel/cl-cookieproject>) - a project skeleton for a ready-to-use project with an entry point and unit tests. With a `src/` subdirectory, some more metadata, a 5AM test suite, a way to build a binary, an example CLI args parsing, Roswell integration.

Credits

- <https://wiki.debian.org/CommonLisp> (<https://wiki.debian.org/CommonLisp>)

Page source: `getting-started.md` (<https://github.com/LispCookbook/cl-cookbook/blob/master/getting-started.md>)

© 2002–2026 the Common Lisp Cookbook Project

 [Discover our contributor's Common Lisp video course on Udemy](https://www.udemy.com/course/common-lisp-programming/?referralCode=2F3D698BBC4326F94358)
(<https://www.udemy.com/course/common-lisp-programming/?referralCode=2F3D698BBC4326F94358>)